

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	K.Sandhya	Reg. No.:	192472207
Section / Batch:	2024/SLOT A	Date:	27-07-2026

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address

Password

Mobile Number

using Regular Expressions.

Validation Rules

Email

Must start with an alphabet.

May contain letters, digits, '.', '_' before '@'.

Domain should contain only alphabets.

Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters

At least one uppercase letter

At least one lowercase letter

At least one digit

At least one special character (@, #, \$, %, &, !)

Mobile Number

Must contain exactly 10 digits.

First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email

Strong Password / Weak Password

Valid Mobile Number / Invalid Mobile Number

Section 2: Design a Finite State Automata (FSA) Simulator

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
 - States
 - Input alphabet
 - Transition table
 - Initial state
 - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:

Strings ending with "ab"

Input

abaab

Output

Transition Path:

$q_0 \rightarrow q_1 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2$

Accepted

Section 3: Smart Pattern Matching Engine using Regular Expressions**3. Problem Statement**

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:

Word Search

Prefix Search

Suffix Search

Date Search

Phone Number Search

Hashtag Search

Mention (@username) Search

Example Input

Meeting on 12/09/2026

Call 9876543210

#NLP

@OpenAI

natural language processing

User Menu

1.Search Date

2.Search Phone Number

3.Search Hashtag

4.Search Mention

5.Search Prefix

6.Search Suffix

Expected Output

Display all matching patterns based on user selection.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Experiment 1: Intelligent Email and Password Validator using Regular Expressions

Aim

To develop a Python program using Regular Expressions (Regex) to validate an email address, password, and mobile number according to predefined security rules.

Procedure

1. Import the **re** (Regular Expression) module.
2. Read the email, password, and mobile number from the user.
3. Create regular expression patterns for:
 - o Email validation
 - o Password validation
 - o Mobile number validation
4. Use the `re.match()` function to compare the user input with the patterns.
5. If the pattern matches, display **Valid**; otherwise display **Invalid**.
6. Execute the program and verify the results.

Code

```
import re

# User Input
email = input("Enter Email: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")

# Regular Expression Patterns
email_pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'

password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#%&!]).{8,}$'

mobile_pattern = r'^[6-9][0-9]{9}$'

# Email Validation
if re.match(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")

# Password Validation
if re.match(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")
```

```
# Mobile Validation
if re.match(mobile_pattern, mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

Sample Input

Enter Email: Sandhya_123@gmail.com
Enter Password: Sandhya@123
Enter Mobile Number: 9876543210

Sample Output

Valid Email
Strong Password
Valid Mobile Number

Result

Thus, the Python program was successfully implemented using **Regular Expressions (Regex)** to validate the email address, password, and mobile number according to the given security rules.

Experiment 2: Design a Finite State Automata (FSA) Simulator

Aim

To develop a Python program that simulates a **Deterministic Finite Automata (DFA)** for recognizing strings ending with **"ab"** and displays the transition path along with whether the input string is accepted or rejected.

Procedure

1. Define the DFA states, input alphabet, transition table, initial state, and final state.
2. Accept an input string from the user.
3. Start from the initial state (q0).
4. Read each character of the input string one by one.
5. Move to the next state based on the transition table.
6. Store the sequence of visited states as the transition path.
7. After processing all characters, check whether the current state is a final state.
8. Display the transition path and print **Accepted** if the final state is reached; otherwise print **Rejected**.

Code

```
# DFA Simulator for strings ending with "ab"

# Transition table
dfa = {
    'q0': {'a': 'q1', 'b': 'q0'},
    'q1': {'a': 'q1', 'b': 'q2'},
    'q2': {'a': 'q1', 'b': 'q0'}
}

initial_state = 'q0'
final_states = ['q2']

# Input
string = input("Enter the input string: ")

current_state = initial_state
path = [current_state]

valid = True

for ch in string:
    if ch not in ['a', 'b']:
        valid = False
        break
    current_state = dfa[current_state][ch]
    path.append(current_state)

if not valid:
    print("Invalid Input (Only 'a' and 'b' are allowed)")
else:
    print("Transition Path:")
    print(" → ".join(path))

    if current_state in final_states:
        print("Accepted")
    else:
        print("Rejected")
```

Sample Input

Enter the input string: abaab

Sample Output

Transition Path:

$q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_1 \rightarrow q_1 \rightarrow q_2$

Accepted

Result

Thus, the Python program to simulate a **Deterministic Finite Automata (DFA)** for recognizing strings ending with **"ab"** was successfully implemented. The program correctly displays the transition path and determines whether the input string is **Accepted** or **Rejected**.

Experiment 3: Smart Pattern Matching Engine using Regular Expressions

Aim

To develop a Python-based text search engine using **Regular Expressions (Regex)** to perform pattern matching for words, prefixes, suffixes, dates, phone numbers, hashtags, and mentions.

Procedure

1. Import the **re** module.
2. Accept a text paragraph from the user.
3. Display a menu with different search options.
4. Based on the user's choice, apply the corresponding regular expression.
5. Search for matching patterns using `re.findall()`.
6. Display all matched patterns.
7. If no match is found, display **No Match Found**.

Code

```
import re
```

```
text = """
```

```
Meeting on 12/09/2026
```

```
Call 9876543210
```

```
#NLP
```

```
#Python
```

```
@OpenAI
```

```
@ChatGPT
```

```
natural language processing
```

```
machine learning
```

```
"""
```

```
print("Given Text:")
```

```
print(text)
```

```

# Search Patterns
date = re.findall(r"\d{2}\d{2}\d{4}", text)
phone = re.findall(r"[6-9]\d{9}", text)
hashtags = re.findall(r"#\w+", text)
mentions = re.findall(r"@ \w+", text)

# Prefix Search (Prefix = nat)
words = re.findall(r"\b\w+\b", text)
prefix = [word for word in words if word.startswith("nat")]

# Suffix Search (Suffix = ing)
suffix = [word for word in words if word.endswith("ing")]

# Display Results
print("\n----- SEARCH RESULTS -----")
print("Date:", date)
print("Phone Number:", phone)
print("Hashtags:", hashtags)
print("Mentions:", mentions)
print("Prefix (nat):", prefix)
print("Suffix (ing):", suffix)

```

Sample Input:

Meeting on 12/09/2026
 Call 9876543210
 #NLP
 #Python
 @OpenAI
 @ChatGPT
 natural language processing
 machine learning

Sample Output:

Given Text:

Meeting on 12/09/2026
 Call 9876543210
 #NLP
 #Python
 @OpenAI
 @ChatGPT
 natural language processing
 machine learning

----- SEARCH RESULTS -----

Date: ['12/09/2026']

Phone Number: ['9876543210']

Hashtags: ['#NLP', '#Python']

Mentions: ['@OpenAI', '@ChatGPT']

Prefix (nat): ['natural']

Suffix (ing): ['Meeting', 'processing', 'learning']

Result :

Thus, the Python program for a **Smart Pattern Matching Engine** was successfully implemented using **Regular Expressions**. The program accurately searches and displays matching dates, phone numbers, hashtags, mentions, prefixes, suffixes, and words from the given text.